

Full-Stack Developer — Take-Home Assignment

Collaborative Task Board with Real-Time Status Updates

Level	Intermediate	Estimated Effort	6–10 hours
Submission Window	5 calendar days from receipt	Role	Full-Stack Developer

Tags

■ JavaScript / TypeScript ■ React ■ Node.js ■ Express ■ REST API ■ MongoDB or PostgreSQL ■ JWT Authentication ■ Git ■ Vercel ■ Render

Overview

This assignment simulates a real feature request you'd receive on the job: build a small, working product end-to-end rather than a toy script. You will design and implement a **Task Board** application — think a lightweight Trello — where a signed-in user can create boards, add tasks to columns (e.g. To Do / In Progress / Done), move tasks between columns, and see changes reflected without a full page reload.

We care less about how many features you cram in and more about the quality of what you ship: a clean data model, a sensibly structured API, sane error handling, and a UI that isn't held together with duct tape.

Aim of the Project

- Evaluate your ability to design and connect a frontend, a backend API, and a persistent database as a cohesive system.
- Assess how you structure code, model data, and handle edge cases without being told exactly how.
- See how you make independent technical decisions (routing, state management, schema design) when requirements are intentionally left a little open.
- Confirm you can take a project from local development through to a publicly reachable deployment.

Functional Requirements

- **Authentication** — Users can sign up and log in. Sessions are protected using JWT (or an equivalent token-based scheme). Unauthenticated requests to protected routes must be rejected.
- **Boards & columns** — A logged-in user can create a board containing at least three fixed columns: To Do, In Progress, Done.
- **Tasks** — Users can create, edit, delete, and move tasks between columns. Each task has at minimum a title, an optional description, and a status matching its column.
- **Persistence** — All data survives a server restart (i.e. stored in MongoDB, PostgreSQL, or another real database — not in memory).
- **API** — Expose a documented REST API (routes, methods, expected payloads) that the frontend consumes; do not hard-code data into frontend components.

- **Validation & error handling** — Reasonable input validation on both client and server, with meaningful error responses (correct HTTP status codes, readable messages).

Stretch goals (optional, not required for a passing submission): drag-and-drop reordering, due dates with visual overdue indicators, or real-time sync across browser tabs via WebSockets/SSE. Attempt these only after the core requirements above are solid.

Suggested Steps

- **1. Plan the data model.** Sketch how Users, Boards, and Tasks relate before writing code. Decide your database and justify it in your README (a sentence or two is enough).
- **2. Scaffold the backend.** Set up an Express (or equivalent Node) server with a clear folder structure (routes, controllers, models). Implement auth endpoints first — everything else depends on them.
- **3. Build the core API.** Implement CRUD endpoints for boards and tasks. Test each endpoint independently (Postman, curl, or automated tests) before wiring up the UI.
- **4. Build the frontend.** Set up a React app that authenticates against your API and renders boards/columns/tasks from real API responses. Keep components small and state predictable.
- **5. Wire up the interactions.** Creating, editing, deleting, and moving tasks should all round-trip through the API and persist on refresh.
- **6. Handle the unhappy paths.** Empty boards, failed logins, expired tokens, network errors — show the user something sensible, not a blank screen or a raw stack trace.
- **7. Deploy it.** Deploy the frontend to **Vercel** and the backend (and database, if self-hosted) to **Render**. Confirm the deployed frontend actually talks to the deployed backend — not to localhost.
- **8. Write the README.** Setup instructions, environment variables required, your data model, and any known limitations or shortcuts you took and why.

Technical Requirements

- Frontend: React (Create React App, Vite, or Next.js all acceptable).
- Backend: Node.js with Express (or a comparably minimal framework — ask first if you'd prefer something else).
- Database: MongoDB or PostgreSQL, hosted (Atlas, Neon, Supabase, or Render's managed offerings are all fine).
- Deployment: frontend live on **Vercel**, backend live on **Render**. Both links must be included in your submission and must be reachable when we review it.
- Source control: a public (or shared-access) GitHub repository with a meaningful commit history — we'd rather see 15 small honest commits than one "final version" commit.

Submission Guidelines

- GitHub repository link (frontend and backend may be one repo or two — your choice, note it in the README).
- Live Vercel URL for the frontend.
- Live Render URL for the backend API.

- A short README covering setup, environment variables, architecture decisions, and anything you'd do differently with more time.
- One working test account (or a self-serve sign-up flow) so we can log in and try it without asking you for credentials.

Evaluation Criteria

Area	What we're looking for	Weight
Functionality	Core requirements work end-to-end, data persists correctly.	35%
Code quality	Clear structure, sensible naming, no dead code, reasonable separation of concerns.	25%
API & data design	Sound schema, RESTful routes, proper status codes and validation.	20%
Deployment	Both services are live, connected to each other, and don't break under normal use.	10%
Communication	README clarity, commit history, and how well trade-offs are explained.	10%

Expected Outcome

By the end of this assignment you will have shipped a small but genuinely functional product: a user can register, log in, create a board, add tasks, move them through a workflow, and trust that their data is still there tomorrow. The application will be reachable at two public URLs — a Vercel frontend and a Render backend — that we can open and use without any setup on our end.

Conclusion

This task is intentionally scoped to be finishable in under a day of focused work while still touching every layer of a real full-stack system: authentication, data modelling, API design, frontend state, and deployment. We're not grading you against a hidden "ideal" solution — a smaller feature set that is well-built, well-tested by hand, and clearly explained will score better than a larger one that's fragile. If a requirement is ambiguous, make a reasonable decision, note it in the README, and move on — that judgment call is itself part of what we're evaluating.

Questions about the brief? Reply to the email this was sent with and we'll clarify — asking a good question is a positive signal, not a negative one.